ONBRD-99: Best Practice Summary

> **Series:** ONBRD | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

Overview

This notebook consolidates every actionable best practice from the ONBRD series (notebooks 01-10) into a single reference. Each practice is definitive: it states exactly what to set and why. Use this as a checklist when onboarding a new Dynatrace environment.

Table of Contents

---

1. Environment Setup and Navigation

Source: ONBRD-01

Practice	Setting / Value	Priority
Record your tenant ID	Write down the tenant ID from `https://{tenant-id}.apps.dynatrace.com` on day one	Critical
Bookmark tenant URL	Save `https://{tenant-id}.apps.dynatrace.com` in browser bookmarks	Critical
Use quick search for navigation	Press **Cmd+K** (Mac) or **Ctrl+K** (Windows/Linux) to find anything	Recommended
Pin frequently used apps	Pin Hosts, Problems, Logs & Events, Notebooks, and Workflows to the sidebar	Recommended
Use OAuth 2.0 for new API access	Create OAuth clients in Account Management; use client credentials flow for all new integrations	Critical
Run discovery queries before deploying	Execute `fetch dt.entity.host | summarize count()` to check for existing data before any agent deployment	Recommended
Explore the Dynatrace Hub	Install apps from the Hub to extend functionality before building custom solutions	Optional

2. IAM and Authentication

Source: ONBRD-02

Practice	Setting / Value	Priority
Configure IAM before deploying agents	Set up SAML/SSO and groups **before** inviting users or deploying OneAgent	Critical
Enable SAML 2.0 or OIDC SSO	Configure enterprise SSO via Account Management > Identity & access management > Single sign-on	Critical
Set Name ID Format to email	In your IdP SAML app, set Name ID Format = **Email address**	Critical
Map required SAML attributes	Map `email`, `firstName`, `lastName` from your IdP to Dynatrace	Critical
Keep a break-glass local admin	Maintain at least **one local admin account** with documented credentials in case SSO fails	Critical
Create four standard user groups	**Platform Admins** (Admin), **SRE Team** (Configurator), **Developers** (Operator), **Stakeholders** (Viewer)	Critical
Link IdP groups to Dynatrace groups	Map IdP group membership to Dynatrace groups for automatic provisioning/deprovisioning	Recommended
Use policies for access control	Use environment policies, account policies, and data policies (with Segments) instead of legacy Management Zones	Critical
Use minimal token scopes	Grant only the scopes each token needs; never create all-scope tokens	Critical
Name tokens descriptively	Use pattern `{env}-{purpose}` (e.g., `prod-oneagent-deployment`, `aws-activegate-useast1`)	Recommended
Set token expiration to 90-365 days	Set explicit expiration on every API token to force rotation	Recommended
One token per use case	Create separate tokens per integration so you can revoke one without affecting others	Recommended
Never commit tokens to code	Store tokens in environment variables or secrets managers only	Critical
Prefer OAuth over API tokens	Use OAuth 2.0 client credentials for all new integrations; reserve API tokens for OneAgent and legacy use	Recommended
Test SSO in incognito window	Verify SSO in a private browser before enabling for all users	Critical

3. ActiveGate Deployment

Source: ONBRD-03

Practice	Setting / Value	Priority
Deploy ActiveGate for restricted networks	Required when hosts cannot reach `*.dynatrace.com` on port 443 directly	Critical
Deploy ActiveGate for Extensions 2.0	Required for SNMP, database extensions, VMware, and custom data sources	Critical
Deploy ActiveGate for private synthetic	Required for monitoring internal applications with synthetic tests	Critical
Deploy ActiveGate for cloud integrations	Required for AWS/Azure/GCP metric polling via CloudWatch/Azure Monitor/Cloud Monitoring	Critical
Deploy 2+ ActiveGates per zone in production	Set **replicas: 2** minimum for HA; OneAgents auto-failover between AGs	Critical
Size ActiveGate by agent count	Small (<=500 agents): 2 GB / 2 cores. Medium (500-1500): 4 GB / 4 cores. Large (1500-3000): 8 GB / 8 cores	Critical
Mount `/opt/dynatrace` on dedicated partition	Allocate **20 GB minimum** for production AG installations	Recommended
Use SSD for extensions workloads	Set disk type to SSD when running Extensions 2.0 on the ActiveGate	Recommended
Place ActiveGate in same network zone as agents	AG must be in the same zone as the OneAgents it serves, with outbound HTTPS to `*.dynatrace.com`	Critical
Assign network zones during install	Use `--set-network-zone=<zone>` parameter during AG installation	Recommended
Name PaaS tokens descriptively	Use `{env}-activegate-{zone}` pattern (e.g., `prod-activegate-dmz`)	Recommended
Use Dynatrace Operator for K8s ActiveGate	Deploy via DynaKube CR with `apiVersion: dynatrace.com/v1beta5` or `v1beta6`	Recommended
Set pod anti-affinity for K8s AG	Use `requiredDuringSchedulingIgnoredDuringExecution` with `topologyKey: kubernetes.io/hostname`	Critical

Practice	Setting / Value	Priority
Create PodDisruptionBudget for K8s AG	Set `minAvailable: 1` to prevent all replicas being evicted simultaneously	Critical
Set resource requests and limits for K8s AG	Small: requests 500m/1Gi, limits 2000m/2Gi. Scale per sizing table	Critical
Use ClusterIP for in-cluster routing	Only use LoadBalancer if external hosts must route through K8s-hosted AG	Recommended
Verify AG health after deployment	Check `curl -k https://localhost:9999/communication/health` and confirm "Connected" in Deployment Status > ActiveGates	Critical

4. Cloud and SaaS Integrations

Source: ONBRD-04

Practice	Setting / Value	Priority
Configure cloud integrations before OneAgent	Set up AWS/Azure/GCP integration first so Smartscape topology includes cloud services from day one	Recommended
Use IAM Role for AWS (not access keys)	Create an IAM role with CloudWatch read permissions and a trust relationship for Dynatrace's AWS account	Critical
Grant least-privilege cloud permissions	AWS: `cloudwatch:GetMetricData`, `tag:GetResources`, `ec2:DescribeInstances`, etc. Azure: **Reader** role. GCP: **Monitoring Viewer** role	Critical
Set AWS polling interval to 5 minutes	Use default 5-minute polling; do not set shorter intervals unless specifically needed	Recommended
Limit AWS regions to those with resources	Only enable regions where you have active workloads to avoid unnecessary API calls	Recommended
Use resource tags to filter monitored resources	In AWS/Azure/GCP settings, use tags to include/exclude resources from monitoring	Recommended
Install extensions from the Hub first	Check Dynatrace Hub for pre-built extensions before building custom ones	Recommended
Assign extensions to an ActiveGate group	Every Extensions 2.0 instance must be assigned to an AG group for execution	Critical
Use OpenTelemetry for direct ingest	For OTel-instrumented apps, send directly to Dynatrace (no ActiveGate required)	Recommended
Verify integration data with entity queries	Run `fetch dt.entity.aws_lambda_function` (or equivalent) to confirm entities are discovered	Critical

5. OneAgent Deployment

Source: ONBRD-05

Practice	Setting / Value	Priority
Use phased rollout	Phase 1: 2-5 non-critical hosts (pilot). Phase 2: one application tier. Phase 3: full production	Critical
Generate dedicated deployment tokens	Create token with `InstallerDownload` scope named `{env}-oneagent-deployment`	Critical
Set host properties during installation	Pass `--set-host-property=env=production --set-host-property=team=platform` at install time	Critical
Set custom host display name	Use `--set-host-name="prod-web-01"` during installation for meaningful names	Recommended
Restart existing processes after install	Restart application servers (Tomcat, JBoss, IIS), web servers, and app runtimes for full code-level visibility	Critical
Enable auto-update	Leave OneAgent auto-update enabled (default) to stay on supported versions	Recommended
Verify deployment with DQL	Run `fetch dt.entity.host | filter state == "RUNNING"` and confirm hosts appear within 5 minutes	Critical
Run connection check on each host	Execute `sudo /opt/dynatrace/oneagent/agent/tools/oneagent-connection-check` after installation	Recommended
Wait 15-30 minutes for full discovery	Allow time for topology, services, and dependencies to fully populate before assessing coverage	Recommended
Use Dynatrace Operator for all K8s deployments	Never install OneAgent manually on K8s nodes; use the Operator exclusively	Critical

6. Kubernetes Deployment

Source: ONBRD-03, ONBRD-05

Practice	Setting / Value	Priority
Use Cloud Native FullStack mode	Set `oneAgent.cloudNativeFullStack` in DynaKube CR; this is the recommended mode for all modern K8s	Critical
Use DynaKube API v1beta5 or v1beta6	Set `apiVersion: dynatrace.com/v1beta5` (or `v1beta6`); v1beta3 is removed in Operator 1.8.0	Critical
Install Operator via Helm	Use `helm install dynatrace-operator dynatrace/dynatrace-operator --namespace dynatrace --set installCRD=true`	Recommended
Create dedicated `dynatrace` namespace	All Dynatrace components go in the `dynatrace` namespace	Critical
Store tokens in K8s Secrets	Create secret `dynakube` with `apiToken` and `paasToken` (or `dataIngestToken`) keys	Critical
Enable `kubernetes-monitoring` capability on AG	Add `kubernetes-monitoring` to `activeGate.capabilities` for cluster API access	Critical
Enable `routing` capability on AG	Add `routing` to `activeGate.capabilities` for OneAgent traffic proxying	Critical
Use namespace selectors for multi-tenant clusters	Set `namespaceSelector.matchLabels` to isolate monitoring per team/tenant	Recommended
Mirror images for air-gapped environments	Pull `dynatrace/dynatrace-operator`, `dynatrace/dynatrace-oneagent`, `dynatrace/dynatrace-activegate` to private registry	Critical
Set tolerations for master/control-plane nodes	Add toleration for `node-role.kubernetes.io/master` with `operator: Exists`	Recommended
Use `--set-host-group` for cluster identification	Pass `args: ["--set-host-group=my-k8s-cluster"]` in cloudNativeFullStack spec	Recommended

7. Organization: Tags, Segments, and Naming

Source: ONBRD-06

Practice	Setting / Value	Priority
Tag at the source	Set host properties, cloud tags, K8s labels, and OTel attributes at the origin; do not rely on Dynatrace-side rule engines	Critical
Define five standard host properties	Set `env`, `team`, `app`, `cost-center`, and `tier` on every OneAgent installation	Critical
Use lowercase, hyphen-separated property keys	Use `cost-center=eng` not `COST_CENTER=eng` or `CostCenter=eng`	Recommended
Use consistent property values	Always `prod` (never sometimes `production`); always `dev` (never `development`)	Critical
Use Segments for data filtering	Create reusable DQL-based Segments in Observe and explore > Segments; do not use legacy Management Zones	Critical
Name segments descriptively	Use `Prod-Checkout-Team` not `Segment1`	Recommended
Establish host naming convention	Use pattern `{env}-{tier}-{seq}` (e.g., `prod-web-01`) and set via `--set-host-name`	Recommended
Leverage cloud tags automatically	AWS tags appear as `aws.tag.*`, Azure as `azure.tag.*`, GCP as `gcp.label.*`; ensure cloud resources are tagged consistently	Recommended
Use K8s labels for container organization	Filter by `k8s.namespace.name`, `k8s.deployment.name`, and `k8s.pod.labels.*` in DQL	Recommended
Document naming conventions	Write down property key/value standards and share with all teams before deployment	Critical

8. Data Exploration and DQL

Source: ONBRD-07, ONBRD-08

Practice	Setting / Value	Priority
Always specify a time range on fetch	Use `fetch logs, from:-1h` not bare `fetch logs`; default 2h scans excessive data	Critical
Filter early in the pipeline	Place `filter` immediately after `fetch`; never `sort` or `summarize` before filtering	Critical
Select fields early	Use `fields` or `fieldsKeep` after filtering to reduce memory; drop unneeded columns	Recommended
Use `==` for exact matches	Use `==` (not `~`) when the full value is known; pattern matching is slower	Recommended
Use `{"a", "b"}` array syntax	Use curly braces and double quotes; never `('a', 'b')` SQL-style	Critical
Alias all aggregations	Write `summarize c = count()` then `sort c desc`; bare `count()` cannot be referenced in `sort`	Critical
Use `timeseries` for metrics	Never `fetch dt.metrics`; always `timeseries avg(dt.host.cpu.usage), from:-1h`	Critical
Convert timeseries arrays to scalars	Use `fieldsAdd val = arrayAvg(ts)` before `filter` or `sort` on timeseries results	Critical
Check for null before aggregating	Add `filter isNotNull(field)` before `summarize` on optional fields	Recommended
Use `entityName()` for display names	Write `entityName(dt.entity.host, type:"dt.entity.host")` to resolve IDs to names	Recommended
Run discovery queries after deployment	Execute entity counts (`fetch dt.entity.host | summarize count()`), service discovery, and log volume queries within 30 minutes of deployment	Recommended
Know the Grail data types	Logs (`fetch logs`), Spans (`fetch spans`), Metrics (`timeseries`), Events (`fetch events`), Problems (`fetch dt.davis.problems`), Entities (`fetch dt.entity.*`)	Recommended

9. Alerting and Workflows

Source: ONBRD-09

Practice	Setting / Value	Priority
Use Workflows for all alerting	Configure alerts in Automate > Workflows; do not use legacy Alerting Profiles	Critical
Create a Davis problem trigger workflow first	Set trigger to "Davis problem" with event = problem opens; this is the minimum viable alerting setup	Critical
Route alerts by team using conditions	Use JavaScript conditions like `event["affected_entity_ids"].some(id => id.includes("checkout"))` to send to team-specific channels	Critical
Create one workflow per team/routing need	Separate workflows per team for independent maintenance and condition tuning	Recommended
Connect Slack via OAuth first	Go to Settings > Integration > Slack and complete OAuth before adding Slack actions to workflows	Recommended
Set up PagerDuty for critical path	Configure PagerDuty integration key in Settings > Integration > PagerDuty for on-call paging	Critical
Use Jinja2 templates in messages	Include `{{ event["title"] }}`, `{{ event["event.category"] }}`, and `{{ event["problem_url"] }}` for dynamic context	Recommended
Configure both open and close notifications	Add triggers for problem opens AND problem closes so teams know when issues resolve	Recommended
Test every workflow before relying on it	Use the workflow test feature to verify delivery, routing, and formatting	Critical
Monitor workflow execution history	Check Automate > Workflows > Executions regularly for failed runs	Recommended
Document escalation procedures	Write down who gets paged, when to escalate, and how to acknowledge across all teams	Critical

10. Dashboards and Visualization

Source: ONBRD-10

Practice	Setting / Value	Priority
Use Dashboards for monitoring, Notebooks for investigation	Dashboards auto-refresh for NOC/status screens; Notebooks for ad-hoc analysis and RCA	Critical
Build an Executive Summary dashboard first	Include KPI row (uptime, errors, avg response, active problems), trend chart, problem list, service health tiles	Critical
Use Single Value tiles for KPIs	Display request count, error rate, host count, active problem count as single-value tiles	Recommended
Use `round(value, decimals: 2)` in dashboard queries	Round percentages and rates to 2 decimal places for clean display	Recommended
Set a default Segment on every dashboard	Apply an environment or team segment so viewers see only relevant data	Recommended
Set auto-refresh interval	Configure refresh rate (30s for NOC, 5m for status boards) in dashboard settings	Recommended
Create three standard dashboards	**Executive Summary** (KPIs + problems), **Service Dashboard** (requests, errors, P95, endpoints), **Infrastructure Dashboard** (host count, CPU, memory, top consumers)	Recommended
Match visualization to data type	Single number = Single Value. Trend = Line Chart. Distribution = Pie Chart. Ranked list = Top List. Entity health = Honeycomb	Recommended
Share dashboards with groups, not individuals	Set sharing to user groups (from ONBRD-02) for easier access management	Recommended
Export dashboards as JSON for backup	Use JSON export before major changes; store in version control	Optional

---